Sharding

什麼是分片 (Sharding)？

你的 App 起飛了。流量持續成長，用戶不斷加入，資料庫也越來越大。一開始你靠著升級到更大的 database instance，皆著更多 CPU、記憶體和儲存空間來撐過了一段時間。

但你終究會撞到單台機器的天花板。查詢開始變慢，寫入成了瓶頸，儲存空間逼近上限。就算是 Amazon Aurora 這樣強大的雲端資料庫，也有大約 256 TB 的硬性限制。

當單台 database 跟不上了，你只剩下一條路：

把資料拆散到多台機器上。

這就是 Sharding。它是規模化的必要手段，但同時也帶來了新的挑戰。這篇講義會說明怎麼分片、什麼時候應該分片，以及需要注意哪些陷阱。

「Partitioning」和「Sharding」這兩個詞常被人混用。嚴格說起來它們有點不同：Partitioning 通常指在單一 database instance 內拆分資料；Sharding 則是把資料分散到多台機器上。實際上大多數工程師都不太區分，不用糾結措辭，只要說清楚你的資料是放在一台機器上還是多台就好。

先說什麼是 Partitioning

Partitioning 是在單一 database instance 內部，把一張大表拆分成較小的片段。它不會增加機器，而是讓資料庫以更有效率的方式管理資料。

想像一張有五億筆資料、佔用 2 TB 的訂單表。查詢上個月的訂單必須掃過整張表，索引變得龐大難以維護，而 vacuum、analyze、重建索引這類例行操作可能鎖住整張表，影響效能。

Partitioning 的解法是把這張大表拆成更小的分區。資料還是放在同一台機器上，只是被切成資料庫可以分開管理的邏輯片段。現在查詢上個月的訂單，只需要掃描對應的分區，不用讀整張表。

Partitioning 有兩種常見類型：

水平分區（Horizontal partitioning）：把 row 拆散到不同分區。比如每年的訂單放一個分區——同樣的欄位，但每個分區的 row 數更少。

buildnotes.org

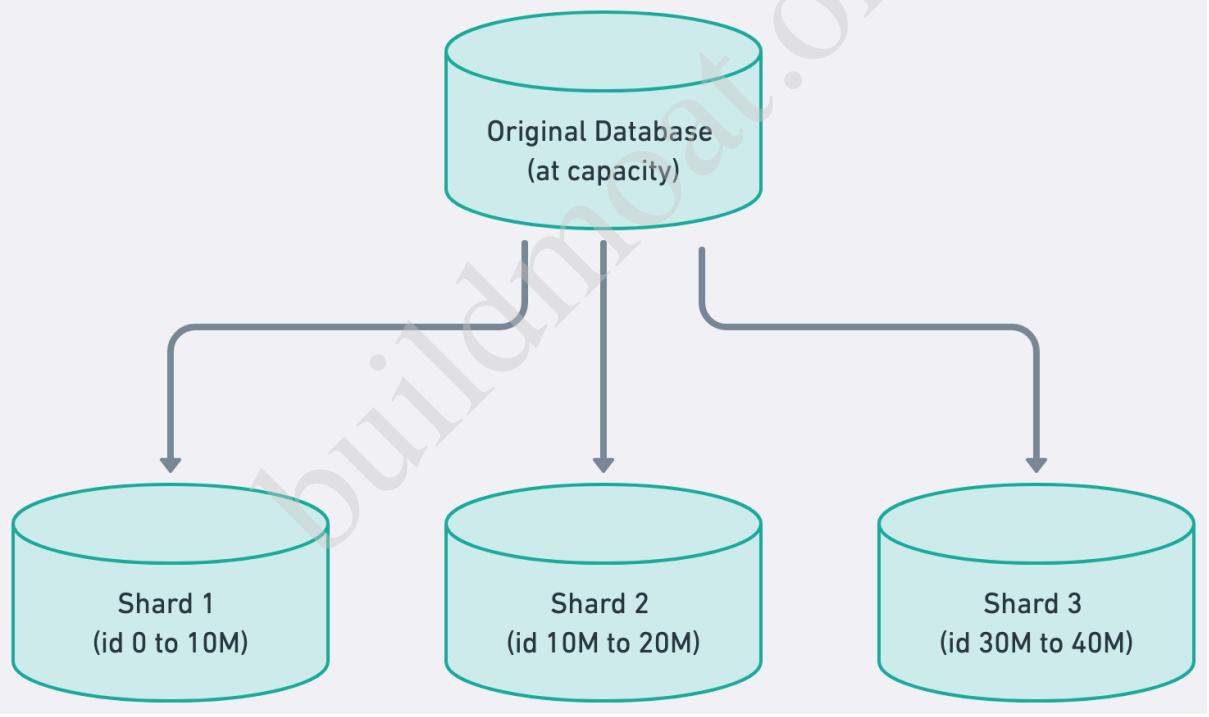
垂直分區（Vertical partitioning）：把 column 拆散到不同分區。比如常用的欄位放一個分區，體積大或很少存取的欄位放另一個——同樣的 row，但每個分區的欄位更少。

什麼是 Sharding

Sharding 是把水平分區的概念延伸到多台機器上。每個 shard 持有一部分資料，所有 shard 合在一起才是完整的資料集。和只在單一 database instance 內拆分的 partitioning 不同，sharding 把負載分散到多台獨立的資料庫上。

舉例來說，如果我們用 `id` 來分割訂單資料，結果大概會長這樣：

- Shard 1 → 訂單 ID 1 ~ 100 萬
- Shard 2 → 訂單 ID 100 萬 ~ 200 萬
- Shard 3 → 訂單 ID 200 萬 ~ 300 萬



buildmoat.org

每個 shard 都是一台獨立的資料庫，有自己的 CPU、記憶體、儲存空間和連線池。沒有任何一台機器持有全部資料或承擔全部流量，這讓你可以隨著增加 shard 數量來擴展儲存容量和讀寫吞吐量。

Sharding 解決了擴展問題，但同時引入了新的挑戰：你需要選擇 shard key、把查詢路由到正確的 shard、避免熱點，還要在 shard 成長時做資料再平衡。接下來我們來談怎麼處理這些問題。

如何拆分你的資料

決定 shard 時，你需要做兩個相互配合的決策：

要用什麼欄位來分片：這個欄位決定資料怎麼被分組。

要怎麼分配這些分組：這個規則決定資料怎麼被分散到各台機器上。

選擇 Shard Key

在面試中，常見的說法是「我要用某個欄位來分片」。關鍵在於你知道要選哪個欄位當 shard key，以及為什麼。

爛的 shard key 會導致資料分佈不均、某個 shard 被猛打而其他 shard 閒置的熱點問題，以及查詢必須打到每台 shard 才能找到資料。好的 shard key 能均勻分散資料、契合你的查詢模式，並且隨著系統成長而能夠擴展。

好的 shard key 需要具備以下條件：

高基數（High cardinality）：key 要有很多不同的值。用布林欄位（true/false）分片最多只能有兩個 shard，根本沒有意義。用 user ID 分片，在有幾百萬用戶的情況下，有足夠的空間分散資料。

均勻分佈（Even distribution）：值要能均勻分散到各個 shard 上。如果用國家分片，而 90% 的用戶都在美國，那個 shard 就會比其他 shard 大得多。User ID 通常分佈很均勻；建立時間戳記在新資料不會全部堆到最新 shard 的前提下也可以用。

契合查詢模式（Aligns with queries）：你最常見的查詢最好只打到一台 shard。如果用 `user_id` 分片，「取得用戶個人資料」或「取得用戶訂單」這類查詢都只打一台 shard。跨所有 shard 的查詢很昂貴。

舉例來說，好的 shard key 像這樣：

🟢 **以 `user_id` 分片的用戶導向 App**：高基數（幾百萬用戶）、分佈均勻，而且大多數查詢本來就限定在單一用戶範圍內（「給我這個用戶的資料」）。完美契合。

🟢 **以 `order_id` 分片的電商訂單表**：高基數（幾百萬筆訂單）、查詢通常限定在特定訂單（「取得訂單詳情」、「更新訂單狀態」），訂單也會隨時間均勻分散。

爛的 shard key 則像這樣：

🔴 `is_premium`**（布林值）**：只有兩個可能值，意味著只有兩個 shard。一個 shard 放所有付費用戶，另一個放免費用戶。如果大多數用戶是免費的，那個 shard 就過載了。

🔴 **成長中的表使用** `created_at`：所有新的寫入都打到最新的 shard，那個 shard 成了寫入熱點，而舊的 shard 幾乎只在讀取歷史資料時才有流量。

bullmeat.org

Sharding 策略

確定了 shard key 之後，你需要決定怎麼把資料分散到各個 shard 上。有三種主要策略，各有不同的取捨。

範圍分片（Range-Based Sharding）

範圍分片是最直觀的做法：把記錄依照連續的值範圍分組。選一個 shard key，例如 `user_id` 或 `created_at`，然後把值的範圍分配給各個 shard。

以 `user_id` 為例，可能長這樣：

- Shard 1 → User ID 1 ~ 100 萬
- Shard 2 → User ID 100 萬 ~ 200 萬
- Shard 3 → User ID 200 萬 ~ 300 萬

範圍分片最大的優點是簡單，而且支援高效的範圍掃描。如果你需要 user ID 50 萬到 60 萬之間的所有訂單，只需要打一個 shard。

但現實中的存取模式通常不會均勻分佈在各個範圍裡。如果用 `created_at` 分片，幾乎所有流量都會打到最新的 shard，因為用戶關心的是近期訂單。新的寫入也只去最新的 shard，舊的 shard 大多閒著。

範圍分片最適合不同用戶自然查詢不同範圍的場景。多租戶系統是個好例子：每家公司拿到一段 ID 範圍——想像一個 SaaS 應用，A 公司的用戶只查 A 公司的範圍，B 公司的用戶只查 B 公司的範圍，負載自然分散到各個 shard 上。

雜湊分片（Hash-Based Sharding）——預設首選

雜湊分片用雜湊函數把記錄均勻分散到各個 shard 上。不是分配範圍，而是把 shard key（如 `user_id`）做雜湊，用結果決定要放到哪個 shard。

以四個 shard 為例，路由方式大概是這樣：

```
shard = hash(user_id) % 4

User 42  → hash(42) % 4 = Shard 2
User 99  → hash(99) % 4 = Shard 3
User 123 → hash(123) % 4 = Shard 1
```

雜湊分片最大的優點是分佈均勻。由於雜湊函數會打散輸入值，新用戶會被均勻分散到所有 shard 上。

缺點在於增減 shard 時。如果從四個 shard 擴展到五個，餘數運算從 `% 4` 變成 `% 5`，幾乎每筆記錄都會對應到不同的 shard，必須移動大量資料。

buildmoat.org

這就是 Consistent Hashing（一致性雜湊）的用武之地。相比簡單的餘數運算，consistent hashing 在增減 shard 時把需要移動的資料量降到最低。

整體而言，雜湊分片是預設也是最常見的策略。在面試中，除非你特別說明用其他策略，面試官通常預設你是用雜湊分片。

目錄分片（Directory-Based Sharding）

目錄分片用一張查找表來決定每筆記錄放在哪裡。不依靠公式，而是把 shard 的對應關係存在一張 mapping 表或服務裡。

```
user_to_shard
--------------
User 15  → Shard 1
User 87  → Shard 4
User 204 → Shard 2
```

目錄分片的強項是靈活性。如果某個用戶產生大量流量，可以把他搬到專屬的 shard 上。如果需要重新平衡負載，只需要更新 mapping 表。你可以實作雜湊函數根本做不到的複雜分片邏輯。

缺點是每個請求都需要一次查找。查詢用戶資料之前，必須先問目錄服務那個用戶在哪個 shard 上，每個請求都增加了延遲，目錄服務也成了關鍵依賴點。如果目錄掛掉，即使所有的 shard 都健康，整個系統也會停擺。

目錄分片適合需要最大靈活性、且能接受額外查找成本的場景。大多數系統從雜湊分片或範圍分片出發，只有在有特殊需求時才用目錄分片。

說實在的，雖然目錄分片對動態場景是個合理的解法，但它在系統設計面試中很少是正確答案。它引入了單點故障，而且對每個請求增加延遲，這會讓面試官追問一連串讓對話偏離軌道的問題。

Sharding 的挑戰

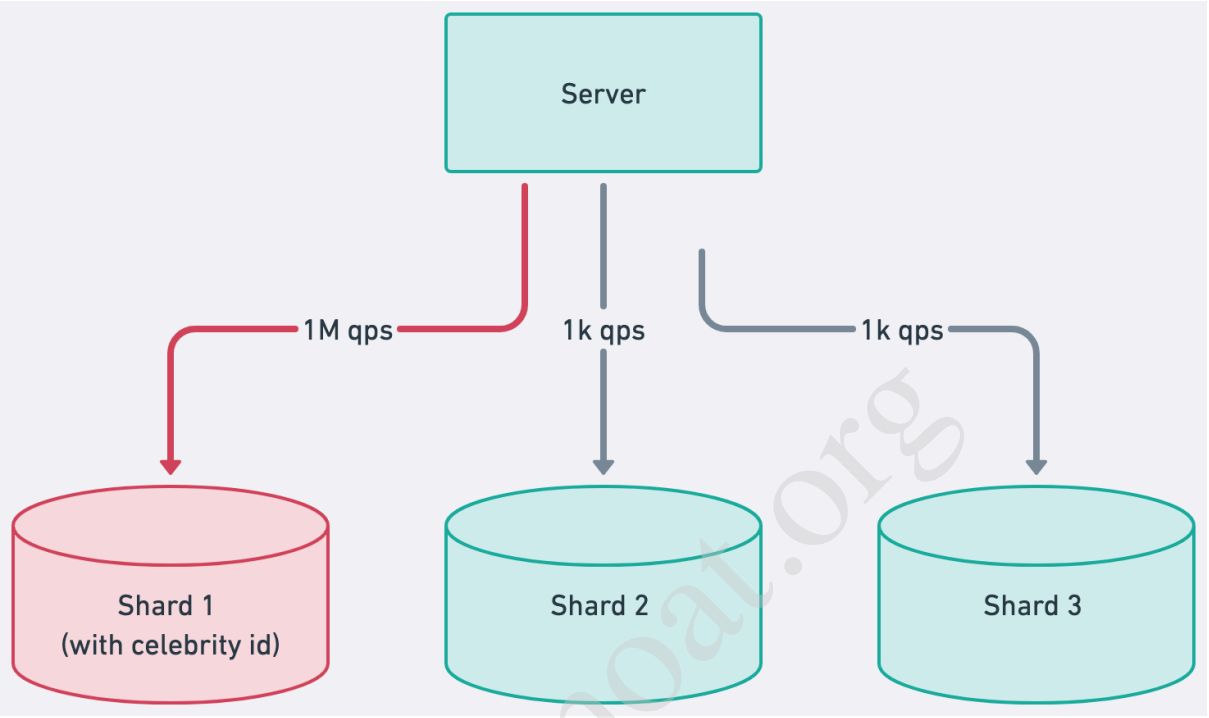
Sharding 解決了擴展問題，但同時引入了新問題。資料現在分散在多台機器上，你必須應對負載不均、跨 shard 查詢，以及跨資料庫的一致性維護。這些挑戰無法完全避免，但只要知道會遇到什麼，就可以在設計上提前因應。

熱點與負載不均

就算有好的 shard key，某些 shard 還是可能收到遠多於其他 shard 的流量。這叫做熱點（hot spot），它抵消了 sharding 的主要好處，因為那台過載的 shard 變成了整個系統的瓶頸。

buildneat.org

最常見的原因是名人問題（celebrity problem）。如果用 `user_id` 分片，Taylor Swift 那個 shard 的流量可能是普通用戶的一千倍。每次有人看她的個人資料、按讚她的貼文或傳訊息給她，請求都打到同一個 shard。這不是分佈策略的問題，雜湊函數對她的 ID 一視同仁，而是有些 key 本來就比其他 key 更活躍。



時間戳記分片則會製造另一種熱點：所有新的寫入都打到最新的 shard，那個 shard 承擔了所有寫入流量，舊的 shard 幾乎只在讀取歷史資料時才有點流量。

偵測熱點的方式是監控各 shard 的指標，例如查詢延遲、CPU 使用率和請求量。當某個 shard 持續表現出比其他 shard 更高的數字，熱點問題就出現了。

應對方式：

把熱 key 隔離到專屬 shard：如果 Taylor Swift 的帳號產生太多流量，把她搬到一個只處理名人帳號的專屬 shard。這也是目錄分片在特定場景下有其用處的原因，不過你不會從一開始就這樣設計。

使用複合 shard key：不只用 `user_id`，而是結合另一個維度，例如 `hash(user_id + date)`。這讓單一用戶的資料隨時間分散到多個 shard，對流量既大又跨越長時間的熱點有幫助。

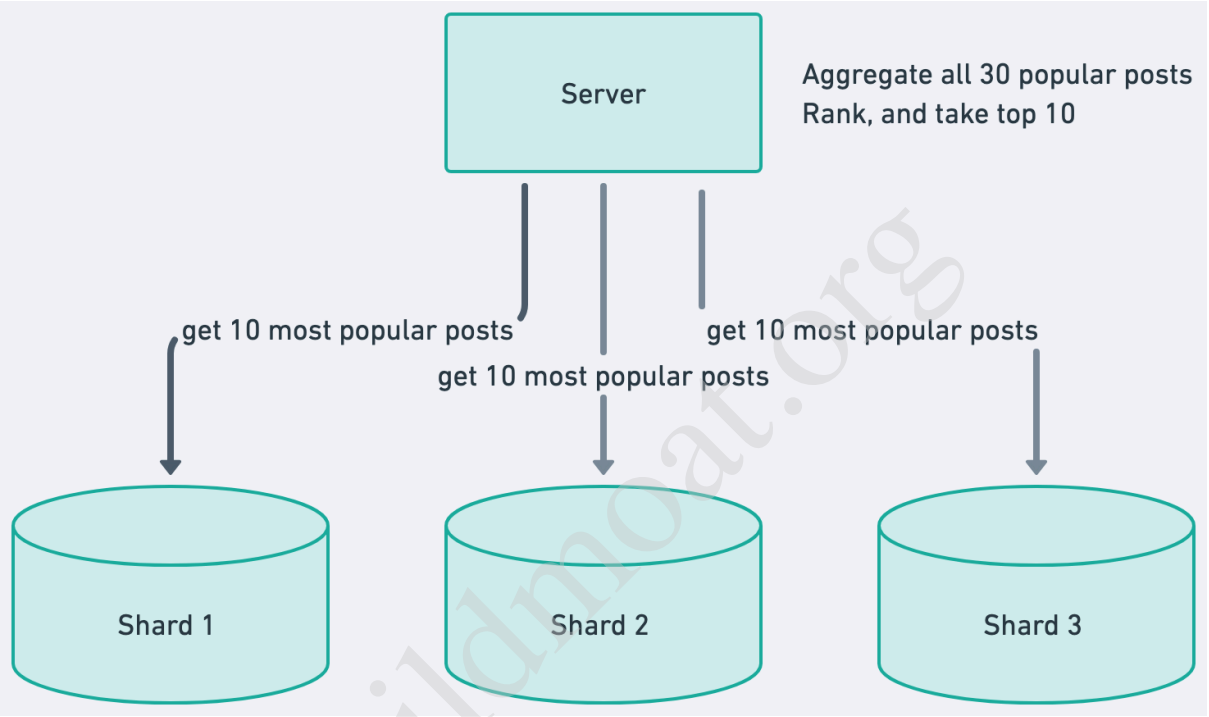
動態 shard 拆分：部分資料庫支援在 shard 太大或太熱時自動拆分。例如 MongoDB 的 balancer 會自動拆分和遷移 chunk 來維持平衡；Vitess 則支援線上 resharding，但需要由操作人員主動發起和管理，不是全自動的。

跨 Shard 操作

buildmoat.org

當資料分散在多台機器上，任何需要從多個 shard 取資料的查詢都會變得昂貴。你必須同時查詢多個 shard，等所有 shard 回應，再自己做聚合。

這個問題在查詢模式和 shard key 不對齊時就會出現。如果用 `user_id` 分片，「取得 user 12345 的個人資料」只打一個 shard，快又簡單。但「取得全站最熱門的 10 篇貼文」就必須查詢每一個 shard，因為貼文散佈在所有用戶 shard 上。你要把查詢發給所有六十四個 shard，等六十四個回應，合併結果，然後才回傳前十名，這是六十四倍的網路呼叫和延遲。



buildmoat.org

你無法完全消滅跨 shard 查詢，但可以把它最小化：

快取結果：如果「全站最熱門的 10 篇貼文」需要打所有 shard，把結果快取五分鐘。第一個查詢很昂貴，但接下來幾千個請求都從快取拿，不需要查六十四個 shard。這在不需要即時準確（可接受最終一致性）的查詢上特別有效，例如排行榜、熱門內容、聚合統計。

反正規化，讓相關資料放在一起：如果你常常需要同時查詢貼文和用戶資料，可以把一些貼文資訊直接存在用戶的 shard 上。是的，這樣資料重複了；是的，更新時更複雜。但它讓你能從一個 shard 拿到所有需要的資料，這個取捨通常是值得的。

接受罕見查詢的代價：有時候一個查詢確實需要打所有 shard，只要它不常發生，這是沒問題的。一個只在一天中開幾次的管理後台顯示「所有 shard 的用戶總數」，跑慢一點完全可以接受。

在面試中，跨 shard 操作往往是設計上需要重新思考的訊號。如果你發現自己對常見的使用場景說「我們查詢所有 shard 然後聚合結果」，暫停一下，想想：能不能反正

規化來避免這件事？能不能快取？能不能用背景任務預先計算？面試官期待你把跨 shard 查詢降到最低，而不是把它當成理所當然。

維護一致性

當你的資料在單一 database 裡，transaction 很直觀。如果你需要同時扣減庫存並建立訂單記錄，把兩個操作包進一個 transaction 就好，要麼都成功，要麼都失敗。資料庫保證一致性。

Sharding 打破了這一點。如果用戶帳號在 shard 1，交易記錄在 shard 2，你就不能用單一 database transaction 了，你在協調兩個完全不知道對方存在的獨立資料庫。

教科書上的解法是兩階段提交（Two-Phase Commit，2PC）：由一個協調者詢問所有 shard 是否準備好提交，等所有人確認就緒後，再要求所有人一起 commit。這保證一致性，但很慢也很脆弱——如果任何一個 shard 或協調者在中途失敗，整個系統可能卡死。大多數生產系統避免 2PC，因為效能和可靠性的代價不值得。

那要怎麼辦？

設計成避免跨 shard transaction：這是最好的解法。如果用 `user_id` 分片，就把一個用戶的所有資料都放在他的 shard 上，像是帳戶餘額、交易紀錄、個人資料，全部放在一起。這樣所有 transaction 都是單 shard 的，快又可靠。

對多 shard 操作使用 Saga 模式：當你真的無法避免跨 shard 協調時，用 Saga 模式。把操作拆成一連串獨立的步驟，每個步驟都有對應的補償動作。如果第三步失敗，就執行第二步和第一步的補償動作來撤銷已完成的工作。這樣做的代價是最終一致性，而不是 2PC 的脆弱性。

以在不同 shard 上的用戶之間轉帳為例：

1. 從 User A 的帳戶扣款（shard 1）
2. 把錢存入 User B 的帳戶（shard 2）
3. 如果第二步失敗，就退款給 User A（補償動作）

接受最終一致性：很多操作不需要嚴格的一致性。如果你在更新一個用戶的粉絲數，而這個數字為了快速查看個人資料而被反正規化存在多個 shard 上，有幾秒鐘各個 shard 顯示不同的數字是沒問題的——最終所有 shard 都會收斂到正確的數字。這比協調分散式 transaction 簡單得多，而且對大多數應用來說，短暫的不一致是可以接受的。

一句話總結：大多數應用都可以被設計成完全避免跨 shard transaction。如果你發現自己一直需要分散式 transaction，很可能是 shard key 選錯了，或者 shard 的邊界劃分有問題。

buildboat.org

現代資料庫中的 Sharding

這裡有個好消息：你大概不會從頭實作 sharding。現代的分散式資料庫大多自動處理 sharding。

常見的 NoSQL 資料庫，例如 Cassandra、DynamoDB、MongoDB，都讓你指定 partition key，其餘的事情由資料庫處理，但它們的底層機制不完全一樣：

Cassandra 使用 partitioner（例如 Murmur3Partitioner）搭配虛擬節點，是 consistent hashing 的一種實作，把 partition key 對應到節點上的 token 範圍。

DynamoDB 對 partition key 做雜湊，把資料路由到內部分區，並隨著資料成長自動拆分和合併分區；這不是對用戶暴露的經典環形 consistent hashing。

MongoDB 依照 shard key 把資料分成以範圍為基礎的 chunk。如果你選擇雜湊 shard key，範圍是在雜湊空間上劃分的；背景的 balancer 會自動拆分和遷移 chunk 來維持 shard 之間的平衡，不是經典的 consistent hashing。

SQL 資料庫在 sharding 這件事上也成熟了很多。Vitess 和 Citus 是流行的開源 sharding 層，架在 PostgreSQL 或 MySQL 前面，處理查詢路由、跨 shard 操作和 resharding，不需要你自己動手。雲端供應商如 AWS Aurora 和 Google Cloud Spanner 則提供內建 sharding 的分散式 SQL。

在面試中，說「我們用 DynamoDB 以 `user_id` 作為 partition key」或「我們用 Vitess 以 `user_id` 分片，需要擴展時由操作人員發起線上 resharding」就已經足夠。除非面試官特別要求，否則不需要實作 sharding 的細節。

系統設計面試中怎麼談 Sharding

Sharding 幾乎在任何討論擴展的時候都會出現。關鍵是知道什麼時候提出來、說什麼，以及避免哪些錯誤。

什麼時候提到 Sharding

小心不要犯過早 shard 的錯誤。你需要先確立為什麼單一 database 不夠用，再來討論 sharding。

在討論容量規劃時碰到以下其中一個限制，就可以帶出 sharding：

儲存空間：「我們有五億用戶，每個用戶有 5 KB 的資料，總共是 2.5 TB。單台 Postgres instance 還可以應付，但如果資料或流量成長了十倍，就需要分片了。」

寫入吞吐量：「我們預期尖峰時每秒有五萬次寫入。單台資料庫撐不住這個寫入量，所以應該要分片。」

讀取吞吐量：「就算有 read replica，如果要服務一億個每天活躍的用戶，每個人發出多個查詢，我們還是需要把讀取負載分散到多個 shard 上。」

buildmoat.org

公式很簡單：**確認瓶頸 → 解釋為什麼單台 database 無法擴展 → 提出 sharding**。

到目前為止我在面試中看到的第一大 sharding 錯誤，就是在還沒證明必要性之前就引入 sharding。慢下來，做算術，確認 sharding 真的是必要的，再開始解釋你怎麼做。

應該說什麼

以下是在面試中用社群媒體 App 為例，逐步說明 sharding 的方式：

第一步：根據存取模式提出 shard key

「這個社群媒體 App 的大多數查詢都是以用戶為中心的。當有人載入他們的 feed，我們在查詢他們的貼文、追蹤對象、按讚紀錄，這些全都限定在單一用戶範圍內。所以我會用 `user_id` 來分片。」

第二步：選擇分佈策略

「我會用雜湊分片搭配 consistent hashing，對 `user_id` 做雜湊，把用戶均勻分散到各個 shard 上。」

第三步：點出取捨

「取捨在於全域查詢會變得昂貴。如果我們需要『所有用戶中的熱門貼文』，就必須查詢所有 shard 並聚合結果。我們可以用快取熱門內容、並用背景任務預先計算來解決這個問題，而不是每個請求都即時計算。」

第四步：說明如何應對成長

「我們從 64 個 shard 起步，留有充裕的成長空間。Consistent hashing 讓之後增加 shard 時比較容易，不需要重新分配全部資料；只有一小部分資料需要移動。」

注意這個流程的自然感，你不只是在列舉事實，而是在說明你的推理過程，並展示你理解其中的取捨。

buildmoat.org

總結

Sharding 是當單台 database 無法應付你的規模時，把資料拆散到多台機器上的做法，藉此提升儲存容量和吞吐量。

有兩個核心決策特別重要：選一個契合你的查詢模式的 shard key，以及選一個能均勻分散負載的分佈策略。選錯了，你就會有熱點問題和昂貴的跨 shard 查詢。

在面試中，當你確認了資料庫瓶頸的時候再提出 sharding。說明你的 shard key 選擇、解釋取捨，並展示你有想到跨 shard 查詢和 resharding 的問題。最重要的是，不要過早分片。一台調校良好的單台 database，能走的路比大多數工程師預期的還要遠。

自我測驗

Q1：Partitioning 和 Sharding 的差異是什麼？

> - **Partitioning**：在同一台 DB 內部將資料邏輯分割（相同機器上的不同區段）。
> - **Sharding**：將資料分散到**不同機器**上（跨機器）。
>
> 實際上很多人混用這兩個詞，重點是說清楚資料是在一台機器還是多台機器上。

Q2：選擇 shard key 時應考慮哪三個條件？為什麼 `is_premium`（布林值）是糟糕的 shard key？

> 三個條件：
>
> 1. **高基數（High Cardinality）**：key 要有很多不同的值。
> 2. **均勻分佈（Even Distribution）**：值要能均勻分散到各 shard。
> 3. **對齊查詢模式（Aligns with Queries）**：最常見的查詢只打到單一 shard。
>
> `is_premium` 是布林值，基數只有 2（true/false），最多只能分成 2 個 shard，完全無法有效分片。

Q3：Range-Based 和 Hash-Based sharding 各自的優缺點？面試預設選哪個？

> - **Range-Based**：優點是支援範圍查詢（如查某段時間的訂單）；缺點是容易產生熱點（如用 created_at，所有新寫入都打到同一 shard）。
> - **Hash-Based**：優點是分佈均勻；缺點是不支援範圍查詢、增減節點時需大量搬移資料（除非用 consistent hashing）。
>
> 面試預設選擇：**Hash-Based**。

Q4：什麼是 Celebrity Problem（Hot Spot）？列舉兩個解決方案。

> **Celebrity Problem**：某些 shard 因為儲存了熱門資料（如名人帳號）而承受不成比例的流量，成為瓶頸。
>
> 解決方案：
>
> 1. **隔離熱 key 到專屬 shard**：把名人帳號搬到只處理名人的專用 shard。
> 2. **使用複合 shard key**：不只用 `user_id`，改用 `hash(user_id + date)` 分散流量。

Q5：如果你發現系統需要頻繁地執行跨 shard 查詢，這代表什麼？

> 頻繁的跨 shard 查詢通常是一個設計訊號，代表 **shard key 選錯了**或 **shard 的邊界劃分有問題**。

buildnoat.org

> 應對方式：重新評估 shard key 選擇、考慮反正規化（denormalization）把相關資料放在同一 shard、對罕見查詢使用快取或背景預計算。

buildmoat.org